Experiment-3.1

Student Name: Gagnesh Kakkar
UID: 23BCS11196
Branch: B.E-C.S.E
Section/Group: 23KRG-2B
Semester: 5th
Date of Performance: 15/10/2025
Subject Name: DAA
Subject Code: 23CSH-301

1. **Aim:** Develop a program and analyze complexity to find shortest paths in a graph with positive edgeweights using Dijkstra‟s algorithm.

2. **Objective:** Code and analyze to find shortest paths in a graph with positive edge weights using Dijkstra‟s

3. **Input/Apparatus Used:** Graph (G =(V,E)) is taken as input for this problem.

4. **Procedure/Algorithm: Pseudocode:**

Follow the steps below to solve the problem:

- Create a set sptSet (shortest path tree set) that keeps track of vertices included in the shortest- path tree, i.e., whose minimum distance from the source is calculated and finalized. Initially, this set is empty.
- Assign a distance value to all vertices in the input graph. Initialize all distance values as INFINITE. Assign the distance value as 0 for the source vertex so that it is picked first.
- While sptSet doesn‟t include all vertices
- Pick a vertex u which is not there in sptSet and has a minimum distance value.
- Include u to sptSet.
- Then update distance value of all adjacent vertices of u.
- To update the distance values, iterate through all adjacent vertices.
- For every adjacent vertex v, if the sum of the distance value of u (fromsource) and weight of edge u-v, is less than the distance value of v, then update the distance value of v.

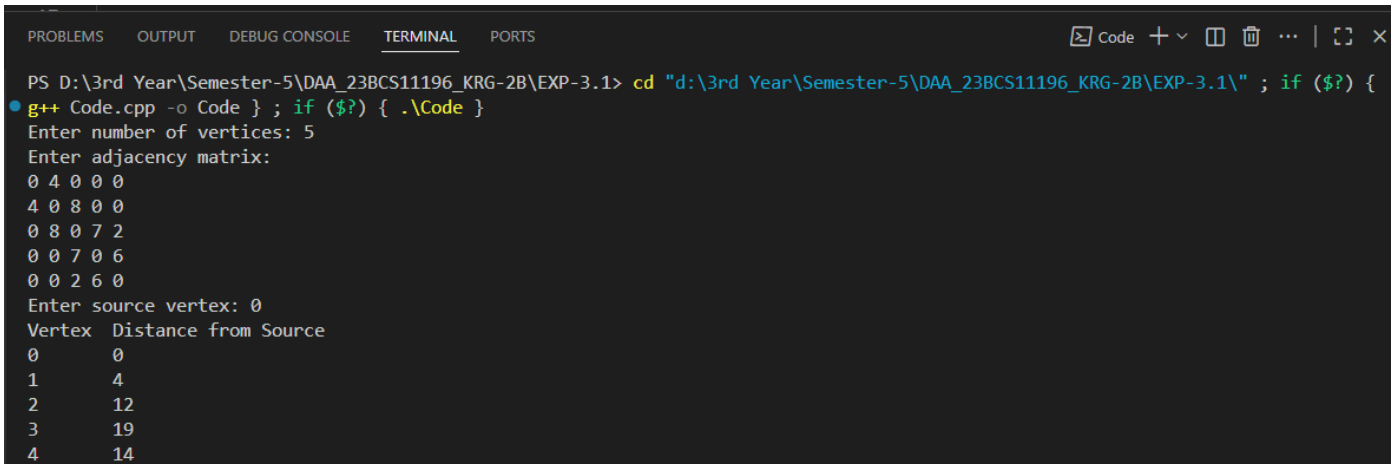
Note: We use a boolean array sptSet[] to represent the set of vertices included in SPT. If avalue sptSet[v] is true, then vertex v is included in SPT, otherwise not. Array dist[] is used to store the shortest distance values of all vertices.

5. Code:

Code.cpp

```cpp
#include <iostream>
using namespace std;

int minDistance(int dist[], bool sptSet[], int n) {
    int min = 999999, index;
    for(int i=0;i<n;i++)
        if(!sptSet[i] && dist[i] <= min) {
            min = dist[i];
            index = i;
        }
    return index;
}

void dijkstra(int graph[20][20], int src, int n) {
    int dist[20];
    bool sptSet[20];

    for(int i=0;i<n;i++){
        dist[i] = 999999;
        sptSet[i] = false;
    }

    dist[src] = 0;

    for(int count=0;count<n-1;count++){
        int u = minDistance(dist, sptSet, n);
        sptSet[u] = true;

        for(int v=0;v<n;v++)
            if(!sptSet[v] && graph[u][v] && dist[u] + graph[u][v] < dist[v])
                dist[v] = dist[u] + graph[u][v];
    }

    cout << "Vertex\tDistance from Source\n";
    for(int i=0;i<n;i++)
        cout << i << "\t" << dist[i] << "\n";
}

int main() {
    int n, src;
    int graph[20][20];

    cout << "Enter number of vertices: ";
    cin >> n;
    cout << "Enter adjacency matrix:\n";
    for(int i=0;i<n;i++)
        for(int j=0;j<n;j++)
            cin >> graph[i][j];

    cout << "Enter source vertex: ";
    cin >> src;

    dijkstra(graph, src, n);
}
```

6. Output:


PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
Code
PS D:\3rd Year\Semester-5\DAA_23BCS11196_KRG-2B\EXP-3.1> cd "d:\3rd Year\Semester-5\DAA_23BCS11196_KRG-2B\EXP-3.1\" ; if ($?) {
g++ Code.cpp -o Code } ; if ($?) { .\Code }
Enter number of vertices: 5
Enter adjacency matrix:
0 4 0 0 0
4 0 8 0 0
0 8 0 7 2
0 0 7 0 6
0 0 2 6 0
Enter source vertex: 0
Vertex Distance from Source
0 0
1 4
2 12
3 19
4 14